BeforeIT.jl: High-Performance Agent-Based Macroeconomics Made Easy

Adriano Meligrana[a], Mitja Devetak[b,c,d], Steven Hoekstra[e], Oliver Reiter[f], Enrico Minnella[g], Sebastian Poledna[h,i], Marco Bardoscia[†g,j], Aldo Glielmo[‡k]

[a] *Sapienza University of Rome, Rome, Italy*
[b] *Paris 1 Panthéon-Sorbonne University, Paris, France*
[c] *Complexity Science Hub, Vienna, Austria*
[d] *Supply Chain Intelligence Institute Austria, Vienna, Austria*
[e] *University of Amsterdam, Amsterdam, The Netherlands*
[f] *Vienna Institute for International Economic Studies (wiiw), Vienna, Austria*
[g] *Bank of England, London, United Kingdom**
[h] *University for Continuing Education Krems, Krems, Austria*
[i] *Austrian Institute of Economic Research (WIFO), Vienna, Austria*
[j] *University College London, Department of Computer Science, London, United Kingdom*
[k] *Banca d'Italia, Directorate General for Information Technology, Applied Research Team, Rome, Italy**

Abstract

BeforeIT is an open-source software for building and simulating state-of-the-art macroeconomic agent-based models (macro ABMs) based on the recently introduced macro ABM developed in [1] and here referred to as the *base model*. Written in Julia, it combines extraordinary computational efficiency with user-friendliness and extensibility. We present the main structure of the software, demonstrate its ease of use with illustrative examples, and benchmark its performance. Our benchmarks demonstrate that the base model built with BeforeIT is orders of magnitude faster than its original Matlab implementation, while achieving performance comparable to a highly optimised HPC implementation. Furthermore, we show how BeforeIT supports an easy construction of model variants such as the 'CANVAS' model of [2], how these variants can be evaluated at scale via an automated multicountry forecasting pipeline, and how the entire simulation can be automatically differentiated. BeforeIT is designed to facilitate reproducibility, extensibility, and experimentation. As the first open-source, industry-grade software to build macro ABMs, BeforeIT can significantly foster collaboration and innovation in the field of agent-based macroeconomic modelling. The package, along with its documentation, is freely available at https://github.com/bancaditalia/BeforeIT.jl under the Apache 2.0 license.

Keywords: software package, agent-based model, simulation, macroeconomics, economic forecasting, automatic differentiation, open-source software, Julia

1. Introduction

Macroeconomic agent-based models (macro ABMs) are computational models in which the micro-level behaviour of heterogeneous interacting agents, such as households and firms, is simulated to produce a macro-level understanding of aggregate economic outcomes [3, 4, 5]. This modelling approach is in contrast with traditional macroeconomic frameworks based on dynamic stochastic general equilibrium (DSGE) models, which solve for equilibrium conditions under well-defined optimisation problems [6, 7, 8] and typically, but not always [9, 10], use representative agents. In comparison to DSGE models, macro ABMs use computational simulations of heterogeneous agents interacting, often requiring greater computational resources and more extensive calibration [11]. On the other hand, ABMs can naturally treat heterogeneity and nonlinear interactions among agents, as well as nonequilibrium dynamics.

Since their emergence in the early 2000s, macro ABMs have become an important tool for analysing economic dynamics, and particularly for reproducing stylised facts and capturing emergent phenomena such as asset bubbles, market crashes, economic divergence and convergence, and technological innovation [12, 4, 13, 14]. ABMs can also provide a strong foundation for policy analysis in areas such as systemic risk, housing market dynamics or inflation, and they have been increasingly employed by policy institutions [12, 15, 16, 17, 5, 18].

The macro ABM developed in [1] represents an important step forward for the field as it demonstrated that

[†] marco.bardoscia@bankofengland.co.uk
[‡] aldo.glielmo@bancaditalia.it
[*] The views and opinions expressed in this paper are those of the authors and do not necessarily reflect the official policy or position of Banca d'Italia or those the Bank of England or its committees.



Figure 1: **Logo of the package.** BeforeIT stands for **B**ehavioural **e**conomic **fore**casting by the **IT** research unit of the Bank of **It**aly. The agents in the 'it' monogram exemplify the agent-based nature of the simulation method. The name of the package also refers to the forecasting possibilities offered by the package, as in predicting a phenomenon *before it happens*.

ABMs, alongside their capacity to reproduce emergent phenomena and stylized facts, can also achieve short-term forecasting performances competitive with vector autoregressive (VAR) and DSGE models. The model's flexibility and detailed representation of economic sectors has enabled it to address a variety of issues, ranging from the COVID-19 pandemic [19] and the 2021–2023 inflation surge [20] to financial crises [21], migration [22], natural disasters [23, 24], and labour productivity losses due to heat stress [25]. The model was extended to work in distributed memory parallel environments in [26], and was taken as the basis for the multi-country macro ABM developed in [27]. Interest in this model has grown among several institutions, including the Bank of Canada, which incorporated it into monetary policy analysis [2, 28, 29].

Over the years, this broad adoption has given rise to multiple implementations of such a model. The original implementation, available in open source, is written in Matlab [1, 30], and it has served as the foundation for other variants. For instance, the Bank of Canada created CANVAS —– a production-oriented modification of the reference Matlab code —– which is currently closed-source. While Matlab is a closed-source language, the original open-source Matlab code can also be run using the open-source Octave language, although this typically implies a significant slowdown in execution time. A Distributed Memory Parallel (DMP-HPC) implementation in C++ using MPI for supercomputers, which is likewise closed-source, has been introduced in [26]. Meanwhile, the University of Oxford [27] and Deloitte [31] have each developed their own closed-source extensions in Python and Java, respectively, underscoring the model's adaptability across a range of environments.

Despite this array of implementations, until now, there was no user- and developer-friendly package that could be freely accessed, installed, and extended for diverse applications. To address this gap, we developed and released BeforeIT, an open-source, thoroughly documented software package that is both user- and developer-friendly, providing computational efficiency and a structure suitable for extensions. BeforeIT takes the model developed in [1] as its core reference, but it also offers a general platform that allows researchers to construct, modify, and extend macro ABMs tailored to their specific applications, thus fostering methodological experimentation and custom analysis. Since the model in [1] forms the foundation of BeforeIT, we refer to it as the base model in the rest of this work.

To ensure quality, reliability, and long-term usability, we made BeforeIT adhere to modern open-source package development standards and best practices. BeforeIT is publicly available on the GitHub page of the Bank of Italy[1], expanding the organization's library of open-source ABM software packages, which includes other models [32] and a calibration engine [33, 34]. Its name and logo, illustrated in Fig. 1, highlight its focus on behavioural agent-based economic forecasting.

Developed in the Julia language, BeforeIT achieves computational efficiency comparable to C-compiled code while preserving readability and maintainability, characteristics typically associated with scripting languages such as Python or Matlab. The overall design of BeforeIT is, in part, inspired by Agents.jl [35], the leading general-purpose agent-based modelling framework in Julia, from which we borrow the idea of a transparent, function-based stepping interface. Julia's balance between ease of use and computational performance has contributed to Julia's growing popularity in economics for numerical computations [36, 37], with notable adoptions by institutions such as the Federal Reserve [38, 39] and the Bank of Canada [40, 41].

By providing a platform to build, run and extend a model of proven impact, BeforeIT fills an important gap in the open-source software community with the aim of fostering innovation, collaboration, and the wider adoption of macro ABMs.

The rest of this work is structured as follows. In Sec. 2 we briefly review the base model as this forms the foundation of BeforeIT, in Sec. 3 we describe the structure of the software and its main features, in Sec. 4 we provide practical examples of software usage, starting from an essential script and moving to more advanced

[1] https://github.com/bancaditalia/BeforeIT.jl

usage such as for scenario analysis, model calibration and automatic differentiation, in Sec. 5 we show how BeforeIT enables the systematic definition and large-scale evaluation of model variants, finally in Sec. 6 we conclude.

2. An essential model summary

The base model, the macro ABM developed in [1], simulates macroeconomic dynamics in a small open economy by modelling the interactions of millions of heterogeneous agents. The model is calibrated using data from national accounts, sectoral data, input-output tables, and business demographics, ensuring that the model reproduces exactly the state of the economy in that quarter in terms of aggregate GDP, GDP components, and industry sizes.

The model adheres to the European System of Accounts (ESA) framework [42] and organises the economy into the following six classes of macroeconomic agents:

- **Households** make consumption, savings, and labour supply decisions.
- **Non-financial corporations** produce goods using labour, capital, and intermediate inputs, with firm-level heterogeneity captured in terms of size, production processes, and market conditions.
- **Financial institutions** mediate credit markets.
- **The central bank** implements monetary policies.
- **The government** is responsible for taxation, public goods provision, and income redistribution
- **The rest of the world** accounts for external trade and international capital flows.

While in the base model the financial sector consists of a single representative bank, BeforeIT generalises it to an arbitrary number of banks, set by the parameter `n_banks`. Each firm and household is randomly assigned to one bank, and each bank keeps its own balance sheet and extends credit subject to its own capital constraint. Setting `n_banks = 1`, the default, recovers the base model.

A distinguishing feature of this model in the literature of macro ABMs is its ability to provide aggregate macroeconomic forecasts — such as for GDP, inflation, and employment — and sector-specific forecasts. The rich micro-structure of the model, combined with adaptive learning, allows agents to form expectations based on past behaviours and observed outcomes. For example, households and firms rely on simple autoregressive (AR(1)) forecasting rules. Meanwhile, decentralised market interactions are characterised by search and matching mechanisms, which introduce trade frictions and market dynamics.

3. Software structure and main features

Initialisation. To instantiate a model, it is sufficient to use its constructor function `Model`. This function takes two inputs: the dictionaries `parameters` and `initial_conditions`. The two dictionaries must be defined using specific keyword strings, such as `"tau_INC"` for the taxation rate on income or `"mu"` for the risk premium over the policy rate charged by commercial banks to issue loans. We use the same keywords used in the original parametrisation, available from the supplementary material of [1]. This ensures full compatibility with the Matlab parametrisations provided by the original paper, allowing users to easily load the Matlab file and use the resulting variables for initialisation. For quick experimentation, we include in the package the original parametrisation for Austria in the first quarter of 2010. We also provide an experimental parametrisation for Italy. However, we believe that the generation of parameters and initial conditions should ultimately be handled in a dedicated package specifically tailored to this purpose.

Structure. The different classes of macroeconomic agents described in the previous section are implemented in BeforeIT as separate structure of arrays (SoA) objects, which in Julia are defined via "structs". Specifically, the package defines the following objects: `w_act` (for households consisting of active workers), `w_inact` (for households consisting of inactive workers), `firms` (for non-financial corporations), `banks` (for the financial sector, composed of one or more banks), `cb` (for the central bank), `gov` (for the government) , `rotw` (for the rest of the world). Each object contains specific and thoroughly documented attributes corresponding to the key features of the respective agent type. For instance, the `w_act` struct includes an attribute `Y_h`, which stores the income of all active workers at any given simulation time-step. The attribute names across all agent types are chosen to align closely with the equations and notation used in the original publication [1] to facilitate a clear connection between the code and the theoretical model.

Finally, the entire model is encapsulated within a struct named `model`, which includes the seven agent

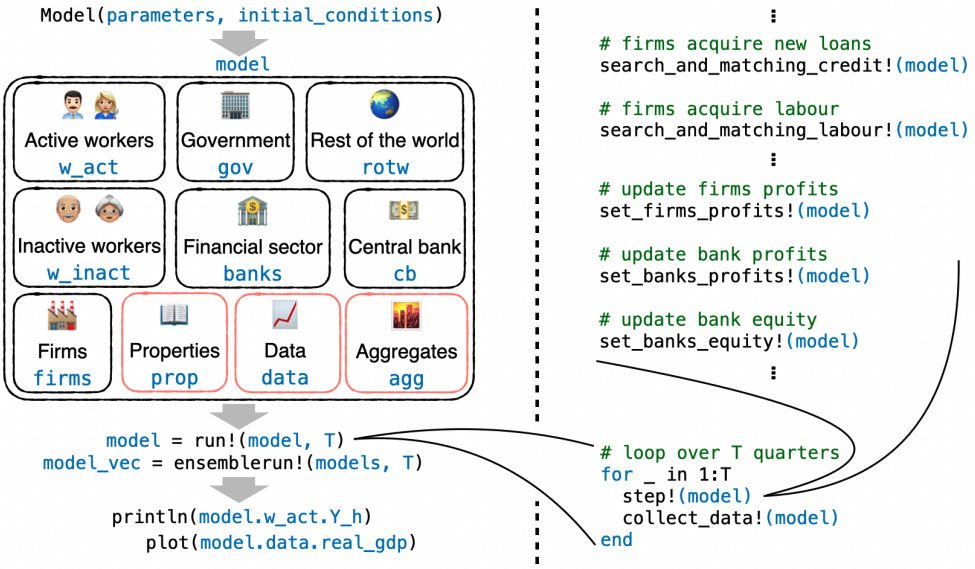


Figure 2: **Illustration of the workflow and main objects of the software**. The constructor `Model` takes as input the two dictionaries `parameters` and `initial_conditions`, returning an object named `model`. This `model` object is composed of distinct objects representing the different agents of the economy, such as `gov` and `firms` and three other objects that do *not* represent agents but store properties and aggregate variables, i.e., `prop`, `data` and `agg`. The `model` object serves as input for `run!`, which iterates over T steps, while a collection of independently initialised `models` serves as input for `ensemblerun!`, which performs `n_sims` independent Monte Carlo runs. Simulation results are stored in the `model` object for a single run, or in the `model_vec` object for multiple runs, both of which allow easy inspection and visualisation of attributes for any agent type as well as of the aggregate variables tracked during the simulation. Similarly, we can inspect any attribute of any of the agent classes. A simulation run simply loops over the function `step!`. One step consists of a series of well-defined function calls updating specific attributes of certain agent types, making up a modular and transparent simulation process.

groups as its attributes, along with three additional objects: the `prop` struct, which stores the properties of the simulation run, the `agg` struct, which holds the aggregate variables representing the overall state of the national economy being simulated, and the `data` struct, which holds the data collected during the simulation run. The left panel of Figure 2 visually illustrates this hierarchical structure. The `model` struct is represented as a large rectangle containing ten smaller rectangles, corresponding to the seven agent classes and the three supplementary objects, `prop`, `data` and `agg`.

Inspection. After a model is created as described above, its attributes can be inspected at any time using the common 'dot' notation. For example, the income vector of all active workers can be printed using `println(model.w_act.Y_h)`, as illustrated in Figure 2. This feature can be especially convenient in interactive scripts or Jupyter notebooks to quickly verify the effects of specific code changes or interventions in the model.

Simulation. Once a model is created using the appropriate function, a simulation can be executed with a single line of code by calling the function `run!`. This function takes the model object as input and returns the updated model after the simulation run. Note that, in Julia, the exclamation mark at the end of a function denotes a function that changes its arguments in-place. In this case, `run!` changes the model in-place, and hence the `model` object after the call stores the final state of the model. In contrast to `step!`, the `run!` function collects data after each step. The `data` object has well-documented, intuitively named attributes, such as `nominal_gdp` or `real_household_consumption`. By default, 32 variables are tracked, five of which are recorded separately for each bank, but the set can be expanded by customising the data tracker. All variables in the `data` object can be inspected using dot notation, for example, by calling `plot(data.real_gdp)`. It is also worth noting that the function for running a single simulation includes a keyword argument, `parallel`, which defaults to `false`. This option allows the consumption goods market for different industrial sectors to run on separate threads, utilising as many threads as are available to the Julia session. It is also straightforward to run

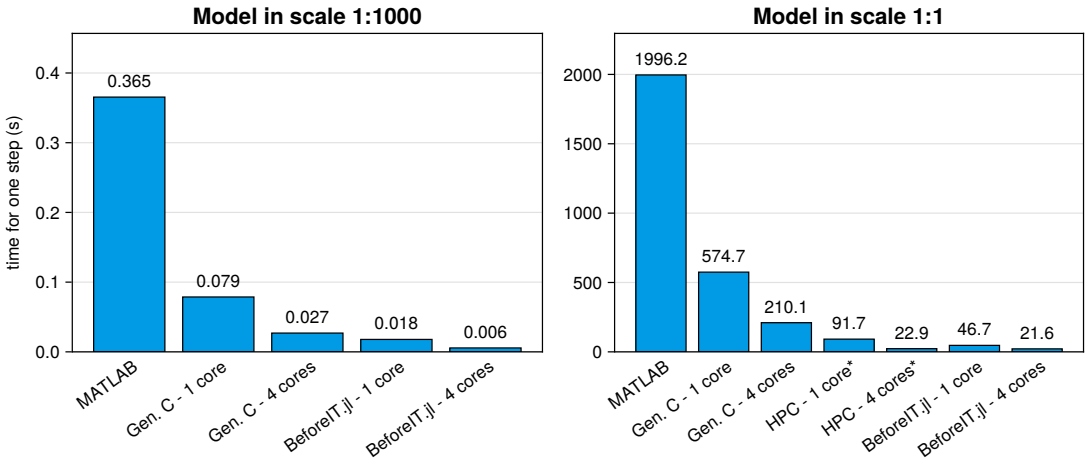


Figure 3: **A benchmark of the computational efficiency for different implementations.** Mean time to run one step of the model calibrated on the Austrian economy with around 8 000 agents and 8 million agents using the original Matlab code, the C code generated by the Matlab Coder toolkit, the HPC implementation and BeforeIT. The benchmarks were executed on Linux x86_64 with an AMD Ryzen 5 5600H CPU and 16 GB of RAM, except for the HPC results, which were only estimated from [26].

a large number of Monte Carlo repetitions of the same simulation using the function `ensemblerun!`. This function takes a collection of models (a vector or a generator) and the number of steps as input and returns a vector of model objects that can also be inspected using dot notation. A keyword parameter, `parallel`, is available also for this function and defaults to `true`. The multi-threading here is more efficient than the intra-model multi-threading across industrial sectors, as the threads operate on independent runs.

Modularity. The right panel of Figure 2 illustrates the inner workings of the model runs. Specifically, the figure shows that the `run!` function is essentially a simple for loop over a desired number of quarters, `T`, of the function `step!`, combined with a function to update the data tracker with the latest model data. The figure also presents an excerpt from `step!`, highlighting how the main simulation loop consists of a sequence of very readable functions. Each function takes the entire `model` as input and updates a specific set of attributes in-place, the trailing exclamation mark again denoting an in-place operation by Julia convention. For example, `set_banks_profits!` takes `model` as input and updates the bank profits stored in the attribute `model.banks.Pi_k`. This clear modular structure allows for precise changes and extensions to the model by simply redefining specific functions.

Speed. In addition to being easy to use and extend, BeforeIT is highly computationally efficient. In Figure 3 we show the efficiency of the software by benchmarking the mean time required to run the model of Austria at scale 1:1000, hence simulating around 8000 agents, and the model at scale 1:1, with around 8 million agents, for one step against the Matlab implementation, the C implementation generated with the Matlab-Coder toolbox and the HPC model developed in [26]. Our tests show that BeforeIT is approximately 20 times faster than the Matlab implementation and 4 times faster than the Matlab-generated C code for the small version in the single-threaded case and, respectively, 43 times and 12 times faster when 8 million agents are simulated. Our simulation appears to have better run-time performance compared to the HPC implementation of the model by looking at the running times reported in the original article [26]. However, given the unavailability of the source code and the different execution environments, we cannot measure the exact speed-up. The efficiency of BeforeIT is largely due to Julia's just-in-time compilation, which enables a performance comparable to C while making the code significantly easier to develop, maintain and understand [43, 44, 45]. Additional performance gains are achieved by employing more optimised algorithms than those used in the original Matlab version. The greatest part of the performance improvement was made possible by using dynamic weighted sampling techniques [46, 47]. In particular, we used the state-of-the-art implementation provided in [48], which allowed to simulate the market exchange process for goods much more efficiently. Notably, even without the use of such advanced algorithms, our software proved to be approximately on par with the C implementation.

```julia
import BeforeIT as Bit
import Plots, StatsPlots

# load parameters and initial conditions
p = Bit.AUSTRIA2010Q1.parameters
ic = Bit.AUSTRIA2010Q1.initial_conditions

# define a simulation length
T = 20

# initialise 8 model objects
models = (Bit.Model(p, ic) for _ in 1:8)

# run 8 simulations in parallel
model_vector = Bit.ensemblerun!(models, T)

# plot the results
plots = Bit.plot_data_vector(model_vector)
Plots.plot(plots...)
```

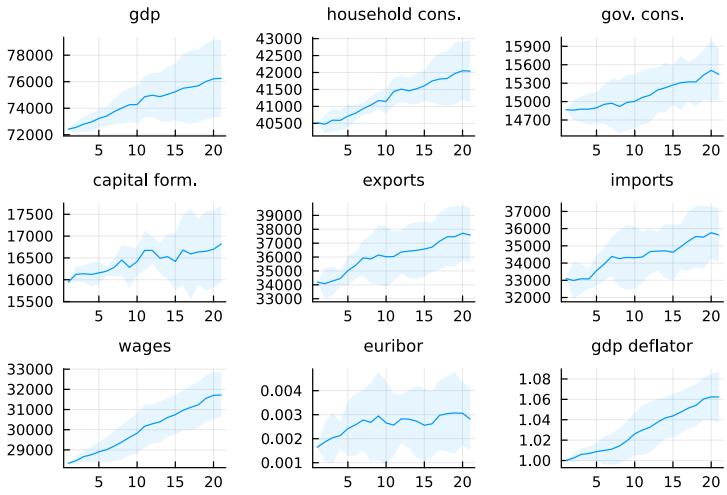


Figure 4: **A simple script and its resulting graphs.** The left panel shows a minimal script to run the model. We import the package, load the original parametrisation for Austria in the first quarter of 2010, and specify a given number of quarters to simulate. Then we call the `Model` function eight times, one for each Monte Carlo run, and finally the `ensemblerun!` function. Illustrative charts of some variables within `model_vector` are shown in the right panel. Solid lines are averages across the eight simulations, while semi-transparent regions span one standard deviation.

The figure also illustrates that applying multi-threading across different industrial sectors can further accelerate the simulation, albeit the speed-up is sub-linear relative to the number of cores employed; even if we believe this aspect could be improved in the future, the scalability of the HPC implementation is still better than what BeforeIT.jl currently achieves. However, when parallelising over different independent Monte Carlo runs, BeforeIT.jl offers the best performance, without sacrificing ease of use.

Testing. BeforeIT includes a large and expanding suite of *unit tests* to ensure its quality and reliability. The current set of unit tests can be divided into three classes. At the lowest level, we test specific functions representing individual agent actions against the expected behaviour. As an example, we test that the function `taylor_rule` with specific values of input arguments returns the same number that one can compute with pen and paper. At the aggregate level, we test that the code respects several accounting identities. For instance, we check that the national income identity holds after every step, i.e., that the GDP equals the sum of all expenditures. Finally, at the model level, we test that a deterministic version of the model exactly matches the detailed behaviour of a deterministic version of the original Matlab code. Additionally, stochastic versions of the two models are rigorously tested for coherence through statistical analysis.

These layers of tests are run automatically through GitHub's continuous integration workflows every time the code is modified to collectively ensure both the accuracy and robustness of the software.

Documentation. BeforeIT is thoroughly documented. The documentation for the main functions governing the inner workings of the package and of the agents in the model is embedded in the source code and also forms an API reference available at https://bancaditalia.github.io/BeforeIT.jl. The webpage provided also includes several tutorials to quickly learn how to use the package with a hands-on approach. The source code reported in the online tutorials is available in the "examples" folder of the GitHub repository along with other example usages.

4. Usage illustration

Here, we present some illustrative examples of how to use the package. We provide self-contained scripts directly here where possible, but we highlight that the complete, runnable source code for all the examples discussed is available in the `examples` folder of the repository[2].

Essential usage. Figure 4 provides a step-by-step example of typical usage of the package. The left panel outlines the exact commands required to load the package, initialise a model, run different Monte Carlo runs of

[2] https://github.com/bancaditalia/BeforeIT.jl/tree/main/examples

the model, and then inspect the results. The right panel presents a few plots of the simulation runs, with the shaded area indicating the standard error on the mean of the Monte Carlo repetitions.

```
# define a custom shock
struct ConsumptionShock <: Bit.AbstractShock
    multiplier::Float64
    final_time::Int
end

# define the shock behaviour
function (s::ConsumptionShock)(model)
    if model.agg.t == 1
        model.prop.psi *= s.multiplier
    elseif model.agg.t == s.final_time
        model.prop.psi /= s.multiplier
    end
end

# load parameters and initial conditions
p = Bit.AUSTRIA2010Q1.parameters
ic = Bit.AUSTRIA2010Q1.initial_conditions

# set shock parameters
my_shock = ConsumptionShock(1.02, 4)

# initialise models
models = (Bit.Model(p, ic) for _ in 1:64)

# run the shocked models
T = 20
model_vec = Bit.ensemblerun!(models, T; shock! =
                                        my_shock)
```

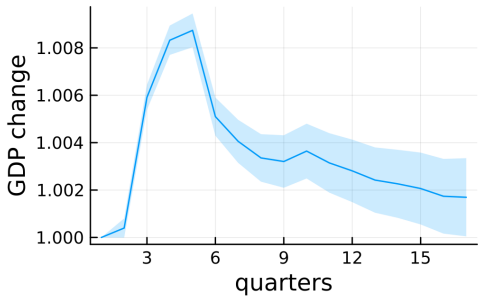


Figure 5: **A script for shocked simulations and its resulting graph.** The top panel illustrates the usage of the interface to run simulations with shocks. One simply defines a callable `struct` and passes it to the function `ensemblerun!` as the `shock!` keyword argument. The effect of the simulated consumption shock on the GDP (ratio of shocked over baseline) is presented in the bottom panel. Solid lines are averages across the 64 simulations, while semi-transparent regions span the standard error.

Shocked simulations. We also provide a high-level interface for simulating arbitrary economic shocks to the model. This functionality is achieved through a "shock" object, implemented as a callable struct.

At the beginning of every step, and before agents' actions, the shock object is invoked. The shock object receives the entire model as input, enabling it to

```
# define a new model object
Bit.@object struct NewModel(Bit.Model)
                        <: Bit.AbstractModel end

# define a new central bank object
Bit.@object mutable struct NewCB(Bit.CentralBank)
                        <: Bit.AbstractCentralBank
    avg_r::Float64
end

# change the default central bank behaviour
function Bit.set_central_bank_rate!(model::
                                    NewModel)
    model.cb.r_bar = model.cb.avg_r + 0.1*randn()
end

# initialise all basic model components
prop           = Bit.Properties(p, ic)
firms          = Bit.Firms(p, ic)
w_act, w_inact = Bit.Workers(p, ic)
banks          = Bit.Banks(p, ic)
gov            = Bit.Government(p, ic)
rotw           = Bit.RestOfTheWorld(p, ic)
agg            = Bit.Aggregates(p, ic)
data           = Bit.Data()

# initialise the custom central bank
cb = Bit.CentralBank(p, ic)
newcb = NewCB(Bit.fields(cb)..., 0.02)

# initialise the model
model = NewModel((w_act, w_inact, firms, banks,
            newcb, gov, rotw, agg, prop, data))

# run 8 copies of the extended model
T = 20
models = (deepcopy(model) for _ in 1:8)
data\_vec = Bit.ensemblerun!(models, T)
```

Figure 6: **A script for model extensions.** The script illustrates the usage of macros and methods' specialisation to extend the model without needing to copy-paste code. One can create a new `mutable struct` for a custom central bank as a subtype of `AbstractCentralBank` and inherit all standard attributes using the `@object` macro. Then, one can modify specific behaviours by specialising some functions on the new model type `model::NewModel`, while the other functions remain unchanged.

modify any model attribute in-place in a highly flexible manner. Figure 5 illustrates this process with an example of a consumption shock. In the figure, we first create a struct with two attributes. Then we make the struct "callable" by defining a function on top of the struct. The function increases the propensity to consume `psi` by the factor `multiplier` until the simulation time `model.agg.t` reaches `final_time`, when it is set back to the original level. Finally, a specific shock (`my_shock`) is instantiated with a 2% increase in the propensity to consume for 4 quarters. This shock is then passed as a keyword argument to the function `ensemblerun!`. By default, the keyword argument is set to a dummy "NoShock" struct that performs no actions. This abstraction allows for arbitrary com-

plex shocks to be implemented straightforwardly since both the state of the model and any of its parameters are accessible as attributes of the `model` object and can be easily manipulated.

Calibration to EU nations. In a separate project called CalibrateBeforeIT.jl [49] we provide functions and exemplary scripts which allow the user to generate the necessary parameters and initial conditions for other EU nations.

The code in the CalibrateBeforeIT.jl package is based on the Matlab and PostgreSQL scripts used in [1] (which were already extended to work for all EU27 countries at that point) and were kindly shared with us. To ease the accessibility of this work, we translated the scripts to Julia, updated Eurostat sources and made the process more robust against missing data. The only prerequisite for this repository is Julia and there are no outside technical dependencies.

We showcase the usage of the package in three scripts (available in the repository). First, usage of the helper functions to a) download data from a data snapshot at Zenodo[3] to be able to replicate our results or b) download the latest available data from Eurostat. The second script shows the application of some necessary data transformations (e.g. appending two data sources to have longer timeseries available). Finally, a third script shows the helper functions that are used to query the data sources for the timeseries necessary to construct the initial conditions and parameters. At our Zenodo repository, we also provide the resulting country-specific calibration objects (which can be used to calculate initial conditions for the BeforeIT model).

At the moment of writing, the CalibrateBeforeIT.jl package works for all EU nations for all quarters from 2015 to 2023.[4]

For quick experimentation, users of BeforeIT can use the `download_zenodo_calibration_object` function to directly download the calibration object of any of the 27 EU member states and calculate `parameters` and `initial_conditions` for a given quarter (specified by the last day of that quarter). A practical example is provided in Figure 7.

We are actively developing the CalibrateBeforeIt.jl package to extend its country and time coverage and remedy any existing inconsistencies.

Automatic differentiation. Automatic differentiation (AD) is a family of techniques for computing derivatives of functions implemented as computer programs by systematically applying the chain rule to the elementary operations performed during execution. In machine learning, AD is widely used for training deep neural networks [50, 51]. More recently, AD and differentiable simulation have been explored for agent-based models, where gradients of simulator outputs with respect to parameters can accelerate tasks such as calibration, optimisation, and sensitivity analysis, provided the model is made suitably differentiable [52, 53, 54].

```julia
using Dates

# download the calibration object for France
co = Bit.download_zenodo_calibration_object("FR")

# calculate initial state for a given time
calibration_date = DateTime(2022, 12, 31)
parameters, initial_conditions =
   Bit.get_params_and_initial_conditions(
      co, calibration_date; scale = 1 / 1000)
```

Figure 7: **Downloading a country calibration object.** The `download_zenodo_calibration_object` function retrieves the pre-computed calibration object for any of the 27 EU member states from the Zenodo data snapshot, here France (`"FR"`). The object is then passed to `get_params_and_initial_conditions` together with a calibration date (the last day of the target quarter) to obtain the `parameters` and `initial_conditions` dictionaries needed to instantiate a `Model`. The `scale` keyword sets the agent-to-population ratio, here 1:1000.

Among the many AD tools available in Julia [55, 56, 57, 58, 59, 60],BeforeIT currently leverages the Mooncake compiler [60] to allow for automatic differentiation. Achieving this compatibility required refactoring specific segments of the codebase to circumvent non-differentiable code blocks. For instance, because Mooncake cannot differentiate through foreign function calls (such as LAPACK routines), the singular value decomposition (SVD) used to estimate expected GDP and inflation had to be re-implemented in pure Julia. While currently unsupported, future integration with Enzyme [58] would be beneficial for cross-validating gradient computations across distinct AD backends.

To verify the effectiveness of using automatic differentiation, we run a simple dummy optimisation on the GDP. Specifically, we tried to optimise the objective

$$\frac{GDP(\theta)_{t=2} - GDP(\theta)_{t=1}}{GDP(\theta)_{t=1}} - \left(\frac{GDP(\theta)_{t=1} - GDP(\theta_0)_{t=1}}{GDP(\theta_0)_{t=1}}\right)^2,$$

where $GDP(\theta_0)_{t=1}$ represents the initial GDP of the original Austrian parametrisation, while $GDP(\theta)_{t=1}$ and $GDP(\theta)_{t=2}$ denote the initial and final GDP of the new parametrisation, respectively. This optimisation tries to

[3] https://doi.org/10.5281/zenodo.18698434

[4] Applying the calibration process for years before 2015 and after 2023 is possible, but more prone to fail due to missing data.

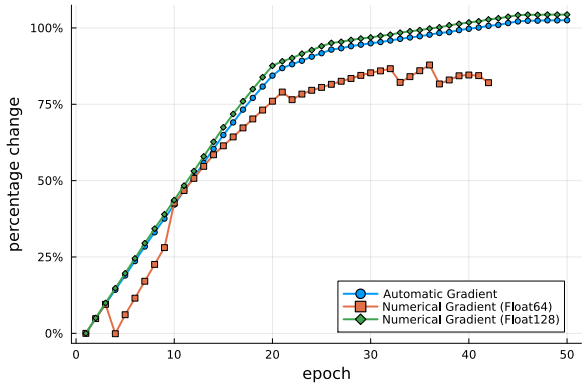


Figure 8: **GDP optimisation trajectories by gradient method.** Percentage change between $GDP(\theta)_{t=2}$ and $GDP(\theta_0)_{t=2}$ along the optimisation, comparing the Mooncake pathwise automatic gradient against central finite differences computed in 64-bit and 128-bit floating-point precision. The 64-bit numerical gradient is too unstable for the optimisation to converge, whereas the automatic gradient tracks the 128-bit numerical gradient closely.

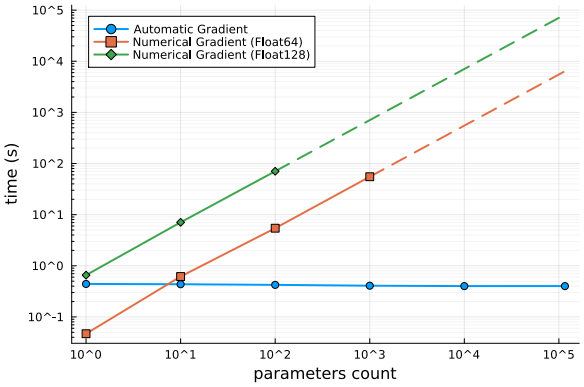


Figure 9: **Performance of gradient computation.** Wall-clock time to compute the gradient of the objective with respect to an increasing subset of the model parameters, comparing the Mooncake automatic gradient against central finite differences.

anchor the GDP at $t = 1$ to the same level of the original parameterisation and, at the same time, induce a greater increase in the change between $t = 1$ and $t = 2$. To make the gradient optimisation more stable, only a subset of 17 core policy and behavioural parameters (which include fiscal rates, household behaviour parameters and banking/financial parameters) have been allowed to vary in the optimisation.

We compared the results of optimising the model through numerical gradients computed using central finite differences and the automatic gradient extracted with Mooncake.jl. Noticeably, using standard 64-bit floating-point numbers for the numerical gradient was too numerically unstable, making the optimisation fail before the end of the optimisation run. Instead, running the optimisation using 128-bit floating-point numbers, made possible by the support of BeforeIT to run the model with any floating-point type, allowed the optimisation to proceed smoothly. Without any adjustment, instead, the automatic gradient ran the optimisation almost as accurately as the 128-bit numerical gradient. It is important to note that the automatic gradient computed through Mooncake is not entirely correct, because Mooncake applies AD to the realised execution trace. When discrete randomness is present in the model (as it is in the search-and-matching markets of our model), the gradient could be biased [61]. In practice, given the results of Figure 8, this does not seem to be the case for our model.

Performance-wise, automatic differentiation changes the scaling of the gradient computation dramatically. As shown in Figure 9, the runtime of the automatic gradient remains essentially flat as the number of active parameters grows from one to the full model parametrisation, at roughly the cost of differentiating one full model step. Instead, central finite-difference gradients scale linearly with the number of parameters, taking about one minute for 1000 parameters, while the automatic gradient remains practical for the full 106,955-parameter state. Thus, automatic differentiation is not only more convenient, but also makes high-dimensional gradient computations feasible.

Extensions. We include in BeforeIT specific macros that allow for easily extending the original agent classes without copying and pasting the entire codebase. In Figure 6 we provide an example of how to leverage these macros. Suppose we want to extend the model by introducing a new type of central bank with custom behaviour. We do this by creating a custom struct called `NewCB`. Crucially, we define this to be a subtype of `AbstractCentralBank`. This allows the new struct to seamlessly inherit all existing functions defined in BeforeIT for central banks.

In our new struct, we want to inherit all existing attributes of the standard central banks in the package and add additional attributes. We do this using the macro `@object`, which essentially copy-pastes all the attributes of the inherited object, which here is `CentralBank`, within the newly created one. Additional attributes can then be defined as needed. Finally, we customise the behaviour of the central bank by defining a new `set_central_bank_rate!` function. In

this function, we specify that the `model` argument is of the custom type `NewModel`. In this way, through Julia's powerful multiple-dispatch mechanism, the compiled code will use this function instead of the default one, and will instead use the default functions for any other behaviours of the central bank as provided in the package implementation.

5. Systematic model improvements

The extension mechanism introduced in Section 4 enables more than ad-hoc experimentation: it provides a principled framework for *systematic* model improvement. By pairing model variants with an automated multicountry forecasting pipeline, researchers can rigorously evaluate alternative modelling assumptions across all calibrated economies using standardised error metrics. This section describes the two main ingredients: (i) how full model variants are defined within BeforeIT, and (ii) how they are evaluated at scale, illustrating the workflow with a concrete example.

Defining model variants. A model variant in BeforeIT is a self-contained Julia file that defines new agent types and overrides specific behavioural functions via multiple dispatch, leaving the rest of the simulation unchanged. The `@object` macro copies all fields from a parent type and appends any additional ones, while the Julia type system ensures that only the overridden functions are replaced. Two representative variants are shipped with the package.

The first, `ModelCANVAS`, implements the demand-pull pricing mechanism of [2] by overriding three functions: `firms_expectations_and_decisions`, which introduces firm-level quantity and price adjustments based on excess demand; `central_bank_rate`, which replaces the fixed-parameter Taylor rule with an adaptive version that re-estimates its coefficients each step; and `growth_inflation_EA`, which records the euro-area time series needed for the adaptive estimation. These overrides require two extended agent types: `CentralBankCANVAS`, which stores the historical interest rate series, and `RestOfTheWorldCANVAS`, which stores euro-area GDP and inflation histories. All other agents (firms, households, banks, government) remain unchanged. We also include two further backwards-compatible improvements to `ModelCANVAS` that allow trajectories to converge over much longer time horizons (around 100 steps). First, we implement Calvo pricing: the parameter `theta_calvo` allows to specify the fraction of firms that can update their prices in each step. Second, we use an exponential moving average with smoothing factor `omega_pi` to compute inflation expectations. `ModelCANVAS` falls back to the behaviour in [2] by setting `theta_calvo` = 1 and `omega_pi` = 0, while typical values for converge over longer time horizons are `theta_calvo` = 0.25 and `omega_pi` = 0.5.

The second variant, `ModelGR`, replaces the log-level AR(1) processes used for expectations and exogenous drivers with growth-rate AR(1) specifications,

$$g_t = \alpha\, g_{t-1} + \beta + \varepsilon_t, \qquad X_t = X_{t-1} \cdot (1 + g_t), \tag{1}$$

where g_t denotes the growth rate of the variable X_t. This affects GDP expectations, inflation expectations, government consumption, and trade volumes, and requires two new agent types, `GovernmentGR` and `RestOfTheWorldGR`, that store the estimated AR(1) coefficients and lagged growth rates. In both cases, the core simulation loop (`step!`) remains completely untouched. Julia's compiler automatically dispatches to the overridden methods based on the model's concrete type, ensuring that every variant is evaluated under identical market-clearing, accounting, and aggregation rules and thereby isolating the effect of the modelling change.

Multicountry forecasting pipeline. To evaluate model variants at scale, BeforeIT includes an automated pipeline that runs out-of-sample forecasting exercises across a list of calibrated countries. A variant is identified by a model constructor, from which the variant name is derived; output is automatically organised under `data/{country}/analysis/{variant}/`, so adding a new variant requires no changes to the pipeline code itself.

The pipeline has two phases, both driven by the same script, shown in Figure 10.

In the *simulation phase*, three helper functions are called in sequence for each country. First, `save_all_params_and_initial_conditions` iterates over all starting quarters in the chosen range (here 2010Q1 to 2019Q4), generating and saving to disk the `parameters` and `initial_conditions` for each quarter. Then, `save_all_simulations` loads each of these, instantiates the model using the provided `model_constructor`, runs `n_sims` Monte Carlo simulations of length `T`, and saves the resulting ensembles. Finally, `save_all_predictions_from_sims` aligns the saved simulations with the real data to produce the forecast trajectories. When `model_constructor` is omitted, the standard `Model` constructor is used, corresponding to the base model.

```
import BeforeIT as Bit
using Dates

# countries to process (2-letter codes)
countries = ["IT", "AT", "ES"]
quarters = DateTime(2010,3,31):Month(3):DateTime
     (2019,12,31)
T, n_sims = 12, 4

# pick the variant; the variant name (and hence the
# output folders) is derived from it, so they agree
model_constructor = Bit.ModelGR
model_variant     = string(nameof(model_constructor))

# --- Simulation phase ---
for country in countries
    cal = Bit.download_zenodo_calibration_object(country)
    folder = "data/$country"
    Bit.save_all_params_and_initial_conditions(
        cal, folder; scale = 0.0005,
        first_calibration_date = first(quarters),
        last_calibration_date  = last(quarters),
    )
    Bit.save_all_simulations(folder; T, n_sims,
        model_constructor,
        simulation_folder = "simulations/$model_variant")
    Bit.save_all_predictions_from_sims(folder, cal.data;
        simulation_suffix = "simulations/$model_variant",
        prediction_suffix = "abm_predictions/$
     model_variant")
end

# --- Analysis phase ---
for country in countries
    cal = Bit.download_zenodo_calibration_object(country)
    Bit.error_table_ar(country, cal.ea, cal.data,
        quarters, Bit.forecast_horizons; model_variant)
    Bit.error_table_abm(country, cal.ea, cal.data,
        quarters, Bit.forecast_horizons; model_variant,
        prediction_folder = "abm_predictions/$
     model_variant")
end
```

Figure 10: **The multicountry forecasting pipeline.** The same script drives the two phases of the pipeline. In the *simulation phase*, the chosen model variant (here `ModelGR`) is calibrated to each country (downloading calibration objects from Zenodo) and run. Forecasted trajectories are saved to disk. In the *analysis phase*, saved forecasts are compared to an AR benchmark using standardised error metrics. One can select a different model variant via `model_constructor`; the output folders automatically reflect the model variant.

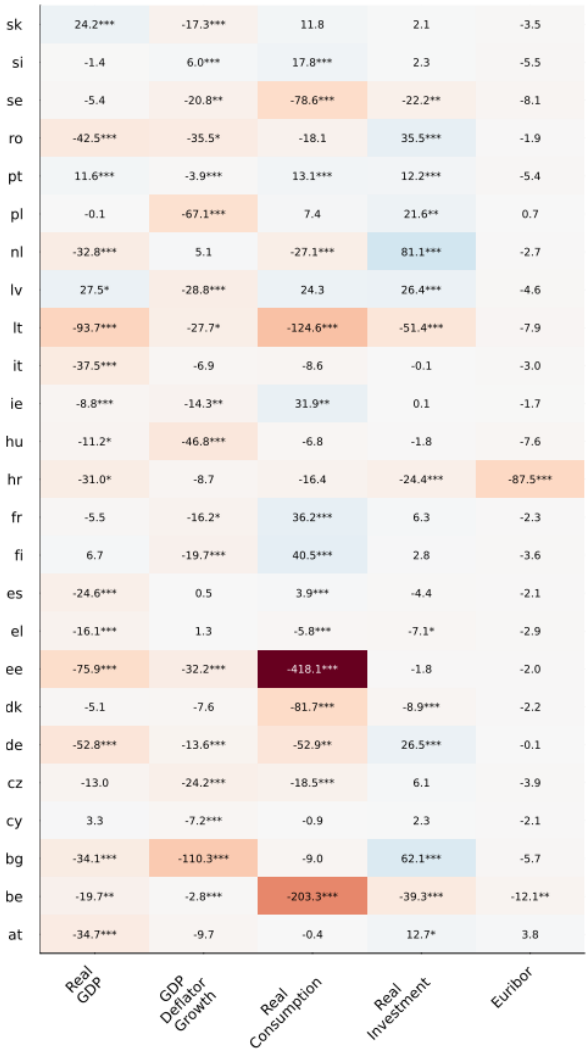


Figure 11: **RMSE improvement of `ModelGR` over a univariate AR benchmark.** Each cell shows $(\text{RMSE}_{\text{AR}} - \text{RMSE}_{\text{ABM}}) \times 100$ over a four-quarter horizon, averaged over all quarters from 2010Q1 to 2019Q4. Blue cells indicate that the ABM produces lower forecast errors than the AR; red cells indicate the opposite. Stars denote significance levels of the Diebold–Mariano, whose null hypothesis is that the two models have equal predictive accuracy ($^{*}p < 0.1$, $^{**}p < 0.05$, $^{***}p < 0.01$).

In the *analysis phase*, forecast accuracy is assessed using the root mean squared error (RMSE) and bias at horizons of 1, 2, 4, 8, and 12 quarters for five key macroeconomic variables: real GDP, GDP deflator growth, real household consumption, real investment, and the Euribor rate. Statistical significance of the bias is tested using a t-test with Newey–West standard errors to account for serial correlation in forecast errors. Model comparisons are performed via the Diebold–Mariano test, which assesses whether the difference in squared forecast errors between two models is statistically significant. Results are stored per country in `data/{country}/analysis/{variant}/` and then aggregated across countries to produce summary tables and heatmaps.

Example results. Figure 11 illustrates the kind of output the pipeline produces, showing the RMSE difference between `ModelGR` and a univariate AR benchmark across a large number of calibrated EU countries. Each cell reports $(RMSE_{\text{AR}} - RMSE_{\text{ABM}}) \times 100$, so that positive values (blue) indicate that the ABM outperforms the AR and negative values (red) indicate the reverse; significance stars denote rejection of equal pre-

dictive accuracy under the Diebold–Mariano test.

The results are heterogeneous across countries and variables. The ABM achieves notably better forecast accuracy for real investment in several countries (e.g., the Netherlands and Bulgaria), while the AR benchmark tends to be more competitive for real GDP and inflation in many economies. Outliers such as Estonia's real consumption reflect episodes of large structural change that are difficult to capture with any fixed specification. This heterogeneity illustrates the value of running the pipeline at scale: patterns that would be invisible in a single-country study become apparent when results are aggregated.

6. Conclusions

This paper introduces BeforeIT, a software package to build state-of-the-art macroeconomic agent-based models (macro ABMs) that are based on the recently introduced base model [1], in a fast, user-friendly, and extensible framework. By leveraging the computational efficiency and flexibility of the Julia programming language, BeforeIT delivers high-performance code that is greatly superior to existing implementations of the base model, such as Matlab or Matlab-generated C code, while maintaining simplicity for users and developers. Its modular structure, thorough documentation, and extensive testing ensure that the package can be easily picked up and used for applications ranging from policy analysis and economic forecasting to any model extension. As the first open-source package to build models like [1], BeforeIT fills an important gap in making the latest macro ABMs more accessible. We believe the package can greatly facilitate experimentation, e.g., involving calibration using large volumes of data, or extensive sensitivity analyses and forecasting. The existing open-source Julia ecosystem for data science and numerical simulation can be of great help for these projects. The Python open-source ecosystem could be equally leveraged as it is possible to bridge the two languages seamlessly using libraries such as PythonCall and JuliaCall [62]. Most generally, BeforeIT can contribute to the foundation of the next-generation tools for macro ABM research.

We foresee several lines of development that could further enhance BeforeIT. One important enhancement is the modularisation and standardisation of the calibration scripts. Unlike the rest of the package, these are not modularly written and currently come in the form of an unpolished research prototype. However, they could be easily improved and, potentially, even be adapted to make them work for any European nation. The resulting code could be integrated within BeforeIT or be released as a dedicated companion package. A second line of development concerns the further inclusion of models built on top of the base model. The package already ships two such variants, the `ModelCANVAS` variant inspired by the Bank of Canada's CANVAS model and the growth-rate `ModelGR` variant described here, and we plan to grow this library of variants in collaboration with the community. Similarly, the multi-bank extension of the financial sector currently features banks that follow identical behavioural rules and differ only in their customer base. Allowing banks to differ more fundamentally, for instance in their lending rates or capital requirements, would make BeforeIT a natural tool for the analysis of systemic risk and macroprudential policy. Finally, BeforeIT is already compatible with automatic differentiation through the Mooncake compiler, which opens the way to gradient-based sensitivity analysis and calibration of the model to aggregate data. A natural next step is to broaden this support to additional AD backends, such as Enzyme, which would allow gradients to be cross-validated across independent implementations.

For any of the mentioned extensions or any other improvement to the package, we greatly encourage open-source contributions to the project in the form of issues, pull requests or feedback.

Acknowledgments

We acknowledge open-source contributions from Peter Reschenhofer (Austrian Institute of Economic Research), Arnau Quera-Bofarull (Macrocosm) and Bence Mérő (Central Bank of Hungary) and we thank Claudia Biancotti (Banca d'Italia), Pietro Terna (University of Turin) and Marco Pangallo (University of Turin) for early feedback on the manuscript. AG would like to thank the following colleagues from Banca d'Italia: Marco Benedetti, Claudia Biancotti and Marco Favorito for their continuous help, support and feedback on the project, Andrea Gentili and Sara Corbo respectively for their help on the design of the package's name and logo. SP acknowledges funding from the Vienna Science and Technology Fund (WWTF) (grant number ESS22-040). The views and opinions expressed in this paper are those of the authors and do not necessarily reflect the official policy or position of Banca d'Italia or those of the Bank of England or its committees.

Individual contributions

AM greatly improved and extended the package including, among the many contributions, performance optimisations, testing infrastructure and support for automatic differentiation; MD improved the software quality of the first version of the package and designed the original extension macros; SH implemented the CANVAS model variant, growth-rate model variant, the forecasting significance tables, cross- and autocorrelation analysis, and the multicountry forecasting pipeline; OR developed the calibration to EU nations and the CalibrateBeforeIT.jl package; EM contributed to the code for multiple banks, testing and results output for multiple banks and conditional forecasting; SP provided the original Matlab code and calibration scripts, and helped finding bugs in the original package; MB designed and implemented the infrastructure for multiple banks and the enhancements to CANVAS; AG conceived and coordinated the project, wrote the first version of the package, and wrote the initial manuscript, later improved with contributions from all authors.

References

[1] S. Poledna, M. G. Miess, C. Hommes, K. Rabitsch, Economic forecasting with an agent-based model, European Economic Review 151 (2023) 104306.

[2] C. Hommes, M. He, S. Poledna, M. Siqueira, Y. Zhang, Canvas: A canadian behavioral agent-based model for monetary policy, Journal of Economic Dynamics and Control 172 (2025) 104986.

[3] L. Hamill, N. Gilbert, Agent-based modelling in economics, John Wiley & Sons, 2015.

[4] H. Dawid, D. Delli Gatti, Agent-based macroeconomics, Handbook of computational economics 4 (2018) 63–156.

[5] R. L. Axtell, J. D. Farmer, Agent-based modeling in economics and finance: Past, present, and future, Journal of Economic Literature 63 (1) (2025) 197–287.

[6] L. J. Christiano, M. Trabandt, K. Walentin, DSGE models for monetary policy analysis, in: Handbook of monetary economics, Vol. 3, Elsevier, 2010, pp. 285–367.

[7] M. Del Negro, F. Schorfheide, DSGE model-based forecasting, in: Handbook of economic forecasting, Vol. 2, Elsevier, 2013, pp. 57–140.

[8] L. J. Christiano, M. S. Eichenbaum, M. Trabandt, On DSGE models, Journal of Economic Perspectives 32 (3) (2018) 113–140.

[9] J. Galí, J. D. López-Salido, J. Vallés, Understanding the effects of government spending on consumption, Journal of the european economic association 5 (1) (2007) 227–270.

[10] G. Kaplan, B. Moll, G. L. Violante, Monetary policy according to hank, American Economic Review 108 (3) (2018) 697–743.

[11] M. Pangallo, R. M. del Rio-Chanona, Data-driven economic agent-based models, in: R. M. del Rio-Chanona, M. Pangallo, J. Bednar, E. D. Beinhocker, J. Kaszowska-Mojsa, F. Lafond, P. Mealy, A. Pichler, J. D. Farmer (Eds.), The Economy as an Evolving Complex System IV, SFI Press, Santa Fe, NM, 2026, pp. 308–341.

[12] A. Turrell, Agent-based models: understanding the economy from the bottom up, Quarterly Bulletin Q4, Bank of England (2016).

[13] G. Dosi, A. Roventini, More is different... and complex! the case for agent-based macroeconomics, Journal of Evolutionary Economics 29 (2019) 1–37.

[14] H. Dawid, D. Delli Gatti, L. E. Fierro, S. Poledna, Implications of behavioral rules in agent-based macroeconomics, in: R. M. del Rio-Chanona, M. Pangallo, J. Bednar, E. D. Beinhocker, J. Kaszowska-Mojsa, F. Lafond, P. Mealy, A. Pichler, J. D. Farmer (Eds.), The Economy as an Evolving Complex System IV, SFI Press, Santa Fe, NM, 2026, pp. 412–448.

[15] G. Catapano, F. Franceschi, M. Loberto, V. Michelangeli, Macroprudential policy analysis via an agent based model of the real estate sector, Temi di Discussione (Working Paper) 1338, Bank of Italy (2021).

[16] A. Carro, M. Hinterschweiger, A. Uluc, J. D. Farmer, Heterogeneous effects and spillovers of macroprudential policy in an agent-based model of the UK housing market, Industrial and Corporate Change 32 (2) (2023) 386–432.

[17] M. Bardoscia, A. Carro, M. Hinterschweiger, M. Napoletano, L. Popoyan, A. Roventini, A. Uluc, The impact of prudential regulation on the UK housing market and economy: Insights from an agent-based model, Journal of Economic Behavior & Organization 229 (2025) 106839.

[18] E. Minnella, A. Pereira, E. Tereanu, The devil in the DeTail: assessing state-contingent tail effects of a releasable macroprudential capital buffer using a parsimonious agent-based framework, Staff Working Paper 1198, Bank of England (2026).

[19] S. Poledna, E. Rovenskaya, J. Crespo Cuaresma, S. Kaniovski, M. Miess, Recovery of the Austrian economy following the COVID-19 crisis can take up to three years, IIASA Policy Brief 26, International Institute for Applied Systems Analysis (IIASA) (2020).

[20] J. Grazzini, C. H. Hommes, S. Poledna, Y. Zhang, Understanding post-pandemic inflation dynamics with a behavioral macroeconomic model of the Canadian economy, SSRN Working Paper No. 4381235 (2023). doi:10.2139/ssrn.4381235.

[21] C. Hommes, S. Poledna, Forecasting economic crises: The Great Recession, the sovereign debt crisis, and COVID-19 in the euro area, Economic Modelling 158 (2026) 107497.

[22] S. Poledna, N. Strelkovskii, A. Conte, A. Goujon, J. Linnerooth-Bayer, M. Catalano, E. Rovenskaya, Economic and labour market impacts of migration in austria: an agent-based modelling approach, Comparative Migration Studies 12 (1) (2024) 18.

[23] G. Bachner, N. Knittel, S. Poledna, S. Hochrainer-Stigler, K. Reiter, Revealing indirect risks in complex socioeconomic systems: A highly detailed multi-model analysis of flood events in Austria, Risk Analysis 44 (1) (2024) 229–243.

[24] S. Hochrainer-Stigler, G. Bachner, N. Knittel, S. Poledna, K. Reiter, F. Bosello, Risk management against indirect risks from disasters: A multi-model and participatory governance framework applied to flood risk in Austria, International Journal of Disaster Risk Reduction 106 (2024) 104425.

[25] C. Kimmich, K. Weyerstraß, T. Czypionka, N. F. Fauster, M. Kinner, E. Laa, L. Mateeva, K. Plank, L. Ulrici, H. Zenz, M. Miess, S. Poledna, Economic impact of labor productivity losses induced by heat stress: an agent-based macroeconomic approach, Climatic Change 178 (3) (2025) 36.

[26] A. Gill, M. Lalith, S. Poledna, M. Hori, K. Fujita, T. Ichimura, High-performance computing implementations of agent-based economic models for realizing 1:1 scale simulations of large economies, IEEE Transactions on Parallel and Distributed Systems 32 (8) (2021) 2101–2114.

[27] S. Wiese, J. Kaszowska-Mojsa, J. Dyer, J. Moran, M. Pangallo,

F. Lafond, J. Muellbauer, A. Calinescu, J. D. Farmer, Forecasting macroeconomic dynamics using a calibrated data-driven agent-based model, arXiv preprint arXiv:2409.18760 (2024). doi:10.48550/arXiv.2409.18760.

[28] D. Coletti, A Blueprint for the Fourth Generation of Bank of Canada Projection and Policy Analysis Models, Staff Discussion Paper 2023-23, Bank of Canada (2023).

[29] M.-A. Gosselin, S. Kozicki, Making it real: Bringing research models into central bank projections, Staff Discussion Paper 2023-29, Bank of Canada (2023).

[30] S. Poledna, et al., Economic forecasting with an agent-based model: Matlab code, https://github.com/iiasa/abm, accessed: 2024-11-27 (2023).

[31] Simudyne, Economic forecasting model, https://docs.simudyne.com/commercial_models/econ_forecast, accessed: 2025-01-13 (2024).

[32] S. Brusatin, T. Padoan, A. Coletta, D. Delli Gatti, A. Glielmo, Simulating the economic impact of rationality through reinforcement learning and agent-based modelling, in: Proceedings of the 5th ACM International Conference on AI in Finance, 2024, pp. 159–167.

[33] M. Benedetti, G. Catapano, F. De Sclavis, M. Favorito, A. Glielmo, D. Magnanimi, A. Muci, Black-it: A ready-to-use and easy-to-extend calibration kit for agent-based models, Journal of Open Source Software 7 (79) (2022) 4622.

[34] A. Glielmo, M. Favorito, D. Chanda, D. Delli Gatti, Reinforcement learning for combining search methods in the calibration of economic ABMs, in: Proceedings of the Fourth ACM International Conference on AI in Finance, 2023, pp. 305–313.

[35] G. Datseris, A. R. Vahdati, T. C. DuBois, Agents.jl: a performant and feature-full agent-based modeling software of minimal code complexity, Simulation 100 (10) (2024) 1019–1031.

[36] Dynare Team, Dynare.jl: A Julia rewrite of Dynare - solving, simulating and estimating DSGE models, https://github.com/DynareJulia/Dynare.jl, accessed: 2024-11-24 (2024).

[37] T. Kockerols, Macromodelling.jl: A Julia package for developing and solving dynamic stochastic general equilibrium models, Journal of Open Source Software 8 (89) (2023) 5598.

[38] M. Del Negro, M. Giannoni, P. Li, E. Moszkowski, M. Smith, The FRBNY DSGE model meets Julia, Tech. rep., Federal Reserve Bank of New York (2015).

[39] M. Del Negro, M. Giannoni, A. Gupta, P. Li, E. Moszkowski, Forecasting with Julia, Tech. rep., Federal Reserve Bank of New York (2017).

[40] Bank of Canada, StateSpaceEcon.jl: A Julia package for working with macroeconomic models, https://github.com/bankofcanada/StateSpaceEcon.jl, accessed: 2024-11-24 (2024).

[41] D. Araujo, Open-sourced central bank macroeconomic models, SSRN Working Paper No. 4755247 (2024). doi:10.2139/ssrn.4755247.

[42] Eurostat, European system of accounts - ESA 2010, Manuals and guidelines 822, Eurostat (2013).

[43] R. Sells, Julia programming language benchmark using a flight simulation, in: 2020 IEEE Aerospace Conference, IEEE, 2020, pp. 1–8.

[44] S. Hunold, S. Steiner, Benchmarking Julia's communication performance: Is Julia HPC ready or full HPC?, in: 2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), IEEE, 2020, pp. 20–25.

[45] W.-C. Lin, S. McIntosh-Smith, Comparing Julia to performance portable parallel programming models for HPC, in: 2021 International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), IEEE, 2021, pp. 94–105.

[46] A. Slepoy, A. P. Thompson, S. J. Plimpton, A constant-time kinetic Monte Carlo algorithm for simulation of large biochemical reaction networks, The Journal of Chemical Physics 128 (20) (2008) 205101.

[47] F. D'Ambrosio, H. L. Bodlaender, G. T. Barkema, Dynamic sampling from a discrete probability distribution with a known distribution of rates, Computational Statistics 37 (3) (2022) 1203–1228.

[48] L. O. Hafner, A. Meligrana, Exact and efficient sampling from dynamic discrete distributions with finite-precision weights, arXiv preprint arXiv:2506.14062 (2025). doi:10.48550/arXiv.2506.14062.

[49] Vienna Institute for International Economic Studies (wiiw), CalibrateBeforeIT.jl: Calibration functions for the BeforeIT.jl ABM, https://github.com/ViennaInstitute/CalibrateBeforeIT.jl, accessed: 2026-10-07 (2026).

[50] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, Nature 323 (6088) (1986) 533–536.

[51] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, Automatic differentiation in machine learning: a survey, Journal of machine learning research 18 (153) (2018) 1–43.

[52] P. Andelfinger, Differentiable agent-based simulation for gradient-guided simulation-based optimization, in: Proceedings of the 2021 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation, 2021, pp. 27–38.

[53] F. Kotthoff, T. Hamacher, Calibrating agent-based models of innovation diffusion with gradients, Journal of Artificial Societies and Social Simulation 25 (3) (2022).

[54] J. Dyer, A. Quera-Bofarull, A. Chopra, J. D. Farmer, A. Calinescu, M. Wooldridge, Gradient-assisted calibration for financial agent-based models, in: Proceedings of the Fourth ACM International Conference on AI in Finance, 2023, pp. 288–296.

[55] J. Revels, M. Lubin, T. Papamarkou, Forward-mode automatic differentiation in Julia, arXiv preprint arXiv:1607.07892 (2016). doi:10.48550/arXiv.1607.07892.

[56] M. Innes, A. Edelman, K. Fischer, C. Rackauckas, E. Saba, V. B. Shah, W. Tebbutt, A differentiable programming system to bridge machine learning and scientific computing, arXiv preprint arXiv:1907.07587 (2019). doi:10.48550/arXiv.1907.07587.

[57] W. Moses, V. Churavy, Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients, in: H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, H. Lin (Eds.), Advances in Neural Information Processing Systems, Vol. 33, Curran Associates, Inc., 2020, pp. 12472–12485.

[58] W. S. Moses, V. Churavy, L. Paehler, J. Hückelheim, S. H. K. Narayanan, M. Schanen, J. Doerfert, Reverse-mode automatic differentiation and optimization of gpu kernels via enzyme, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1–16.

[59] G. Arya, M. Schauer, F. Schäfer, C. Rackauckas, Automatic differentiation of programs with discrete randomness, Advances in neural information processing systems 35 (2022) 10435–10447.

[60] W. Tebbutt, H. Ge, Mooncake: Towards a differentiable general-purpose language, https://github.com/chalk-lab/Mooncake.jl, accessed: 2026-10-07 (2024).

[61] A. Quera-Bofarull, N. Bishop, J. Dyer, D. Jarne Ornia, A. Calinescu, J. D. Farmer, M. Wooldridge, Automatic differentiation of agent-based models, arXiv preprint arXiv:2509.03303 (2025). doi:10.48550/arXiv.2509.03303.

[62] JuliaPy, PythonCall & JuliaCall, https://github.com/JuliaPy/PythonCall.jl, accessed: 2024-12-16 (2024).